PROJECT IN ADVANCED ROBOTICS – What We Tried & How We Improved It

Loai Saadi 211403845

Michel Ghanadry 211401674

Mohammad Mhaniha 315649814

GitHub Repo

0) Test inputs

To ensure fair comparison, we used the same fixed inputs in all experiments:

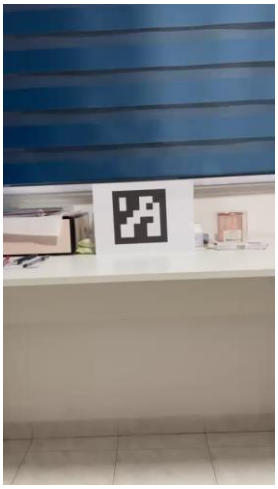

Figure 1: Sample frame #1 used for the two-frame pose estimation (frame_000000.jpg)

Figure 2: Sample frame #2 used for the two-frame pose estimation (paired with Figure 1) (frame_000010.jpg)

1) Goal (what we wanted to achieve)

In this project we wanted to estimate the camera motion between two frames. The output we aimed for is a 6-DoF motion: a rotation matrix R (3x3) and a translation vector t (3x1). After that, we extended the same idea to a full video so we can draw the motion path over time.

- Input: two images OR a video (.mp4)
- Output (two images mode): R (rotation) and t (translation direction)
- Output (video mode): a 3D trajectory plot + rotation over time

2) First working solution (two frames)

We started with a simple and standard Visual Odometry pipeline using ORB features. The idea was: find matching points between the two frames, compute the Essential Matrix, and recover the relative pose between the frames.

- Detect keypoints + descriptors (ORB)
- Match descriptors (BFMatcher)
- Compute Essential Matrix E using RANSAC
- Recover pose using recoverPose -> gives R and t

Small code example (core steps):

```
kp1, des1 = orb.detectAndCompute(img1, None)
kp2, des2 = orb.detectAndCompute(img2, None)

matches = bf.match(des1, des2)
pts1 = np.float32([kp1[m.queryIdx].pt for m in matches])
pts2 = np.float32([kp2[m.trainIdx].pt for m in matches])

E, mask = cv2.findEssentialMat(pts1, pts2, K, method=cv2.RANSAC)
_, R, t, _ = cv2.recoverPose(E, pts1, pts2, K)
```

At this point we already got valid rotation angles and a translation direction. Important note: this t is not in real meters yet - it is only a direction (scale is unknown).

```
============= CAMERA MOTION (6 DoF) =============
Translation (meters, unscaled):
  tx = -0.9966,  ty = -0.0676,  tz = -0.0462

Rotation (degrees):
  rx = -0.02°,  ry = 2.38°,  rz = 0.42°
==================================================
```

Figure 4: Two-frame ORB baseline output (Rotation Matrix R and Translation Vector t).

After these naive trials and retries, we noticed that the two-frame result is not always stable (it can change a lot depending on which two frames we choose). So we moved to a video-based approach to get a smoother and more reliable motion path.

3) Moving from 2 frames to a full video

After the two-frame version worked, we moved to videos. We split the .mp4 into frames and then ran the same two-frame estimation many times: frame i -> frame i+1 (or frame i -> frame i+step).

- We extract frames into a folder (frame_000000.jpg, frame_000001.jpg, ...)
- We estimate motion between consecutive frames
- We accumulate translation to build a trajectory over time

We also tested using a larger step (for example step=5) so we get a bigger motion baseline, which usually improves stability.

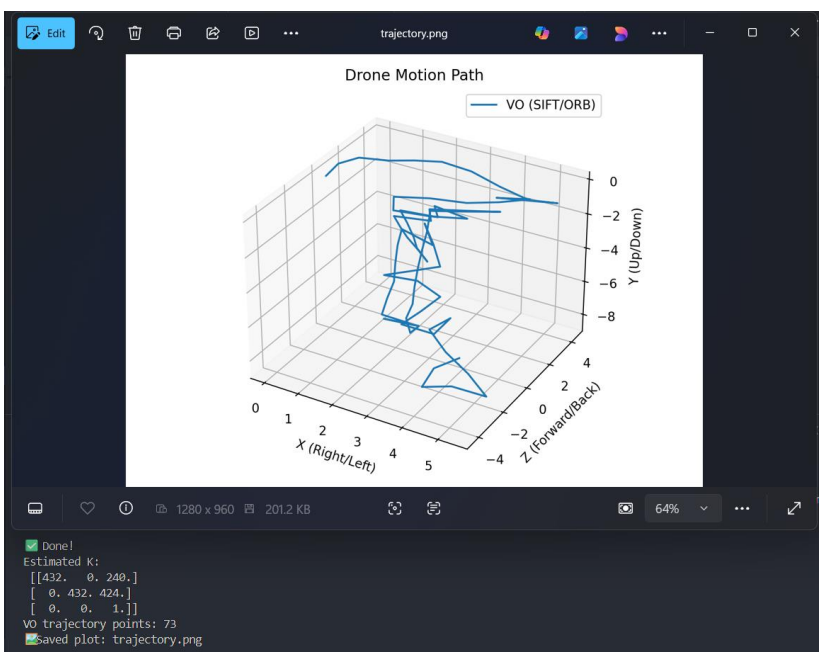

Figure 5: First video-based trajectory plot (baseline) showing why the path was noisy at the beginning.

4) Problems we saw (and why results looked messy)

When we started plotting the 3D path, we noticed the trajectory sometimes looks too noisy or unrealistic even when the real motion is simple (left-right, up-down). This happened for a few reasons:

- Scale ambiguity: VO does not know the real distance without extra info.
- Bad matches: some matched points are wrong, which creates wrong pose estimates.
- Small parallax: if the camera motion is tiny or mostly rotation, the translation is hard to estimate.
- Rolling shutter / stabilization (especially phones): can break the geometric model.

5) What we changed to make the code better

Instead of keeping the first version as-is, we improved it step by step. Each change was small, and we checked if the results became more stable.

5.1 Better matching (less wrong correspondences)

- We replaced simple crossCheck matching with KNN matching + Lowe ratio test.
- We kept only the best matches and ignored weak ones.

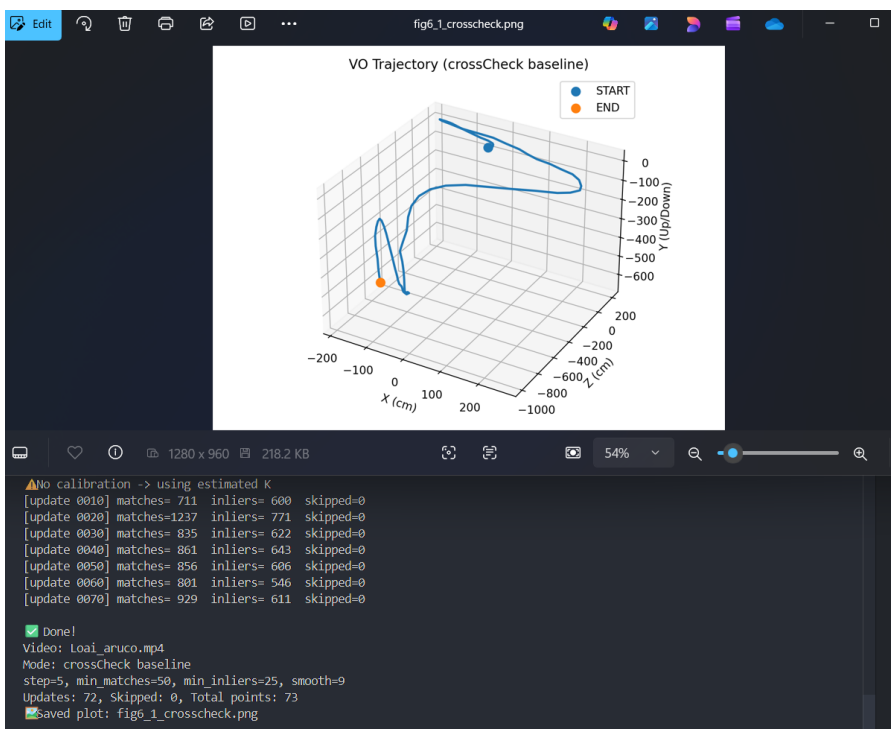

Figure 6.1: Older/simpler matching (before improvement) - more wrong matches and less stable path.

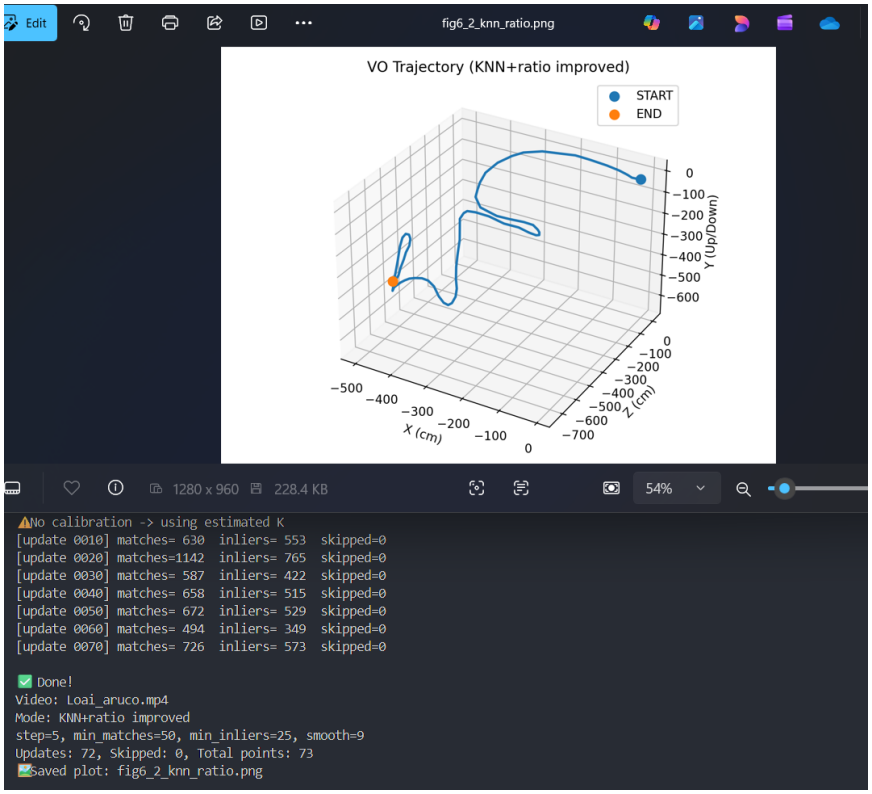

Figure 6.2: Improved matching (KNN + ratio test) - fewer wrong matches and more stable trajectory.

5.2 Switching from ORB to SIFT

We tried SIFT because it is usually more stable than ORB in challenging scenes (lighting / blur / bigger changes). We used FLANN matcher for faster and better nearest-neighbor search.

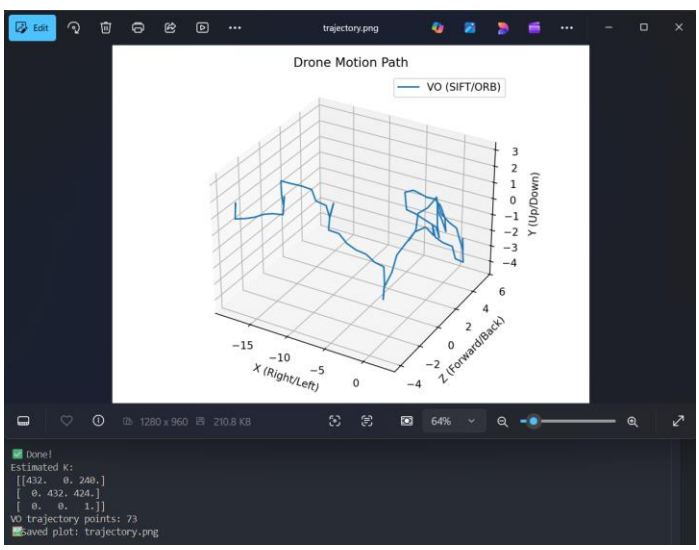

Figure 7.1: ORB trajectory (baseline) - usually noisier on the same video.

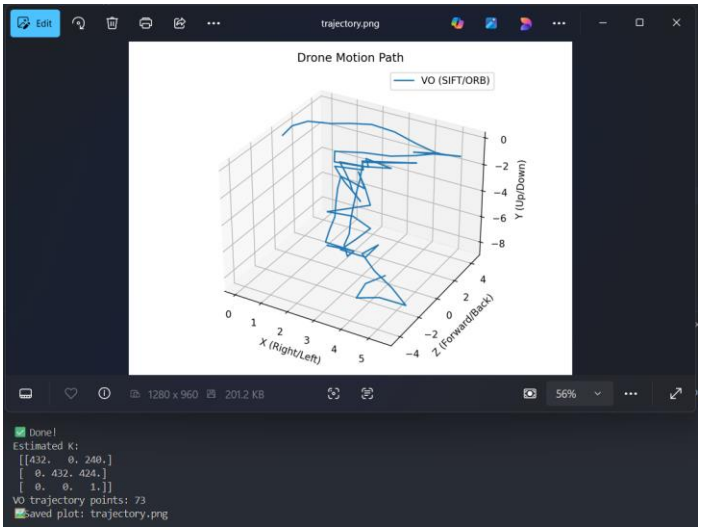

Figure 7.2: SIFT trajectory - more stable on the same video.

5.3 Stronger RANSAC filtering

- We tuned the RANSAC threshold to reject more outliers.
- We used the inlier mask from findEssentialMat to keep only good points.
- We skipped frames when inliers were too few (instead of adding garbage motion).

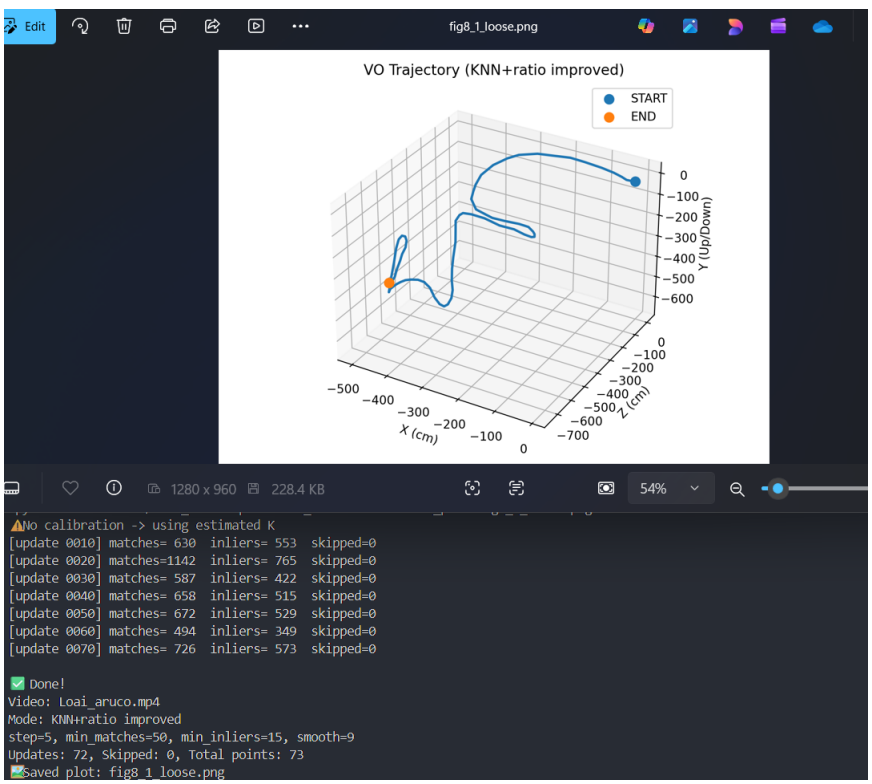

Figure 8.1: Loose filtering - more jumps can appear because weaker updates are accepted.

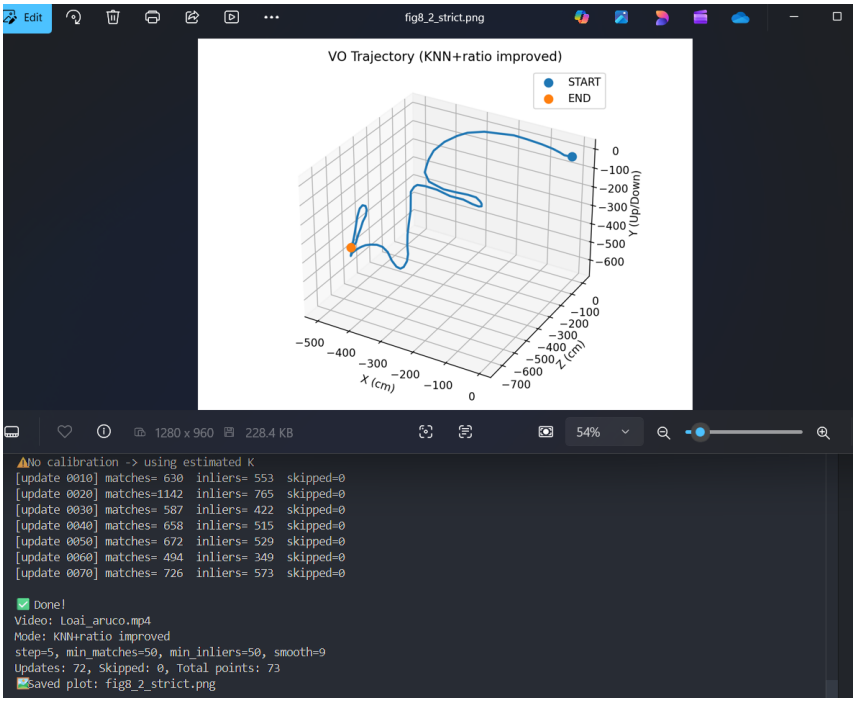

Figure 8.2: Strict filtering - fewer random jumps because bad updates are skipped.

5.4 Smoothing the final path

Even with good matching, small noise is expected. So we applied a moving-average smoothing window on the final trajectory (in centimeters).

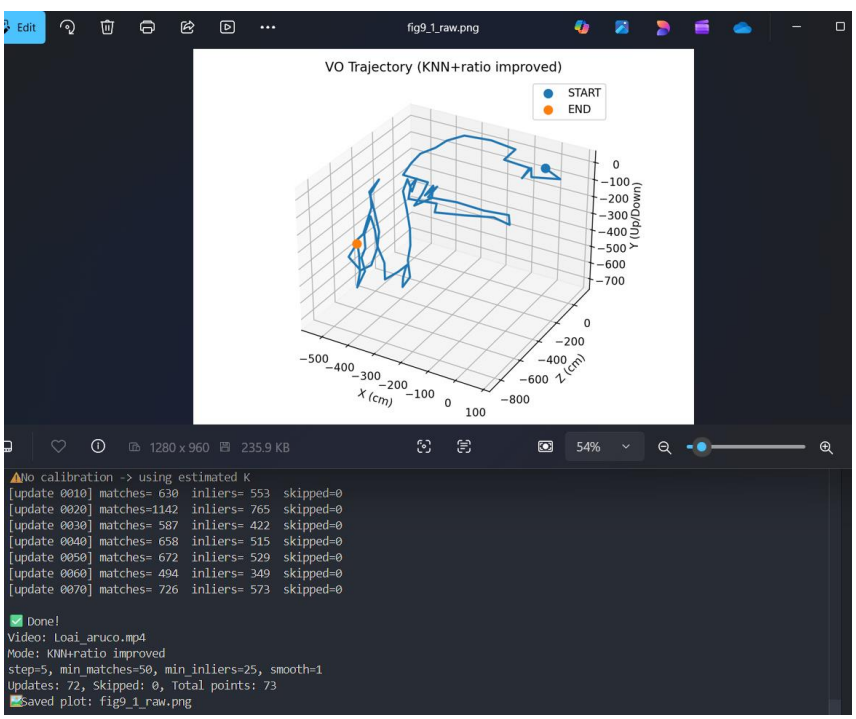

Figure 9.1: Trajectory before smoothing (shows more small jitter).

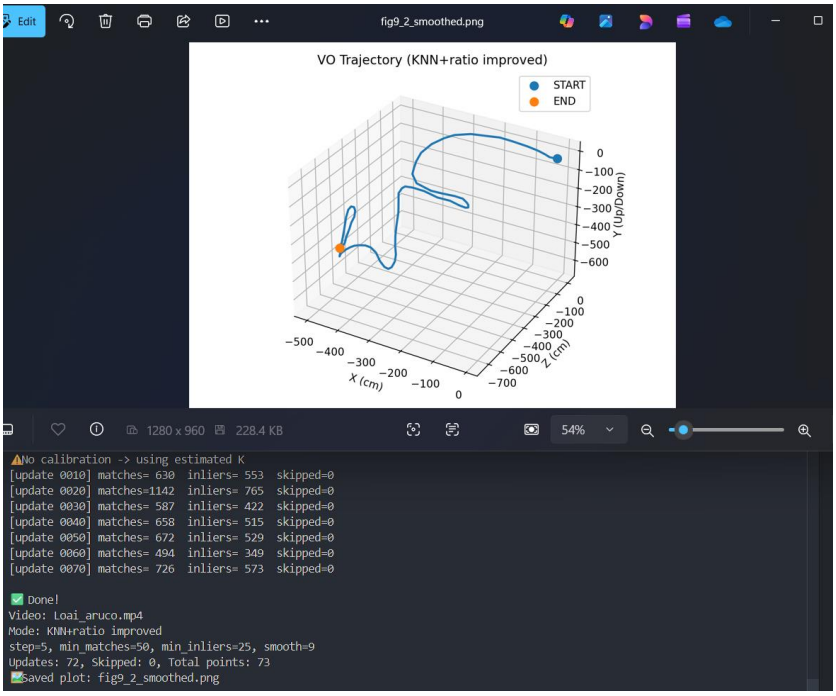

Figure 9.2: Trajectory after smoothing (more stable / cleaner path).

6) ArUco scale and ground-truth idea

To fight the scale problem, we added ArUco markers. When an ArUco marker is visible in a frame, OpenCV can estimate its pose using the real marker size. That gives us a translation in real units (meters), so we can compare VO vs ArUco and even scale the VO translation.

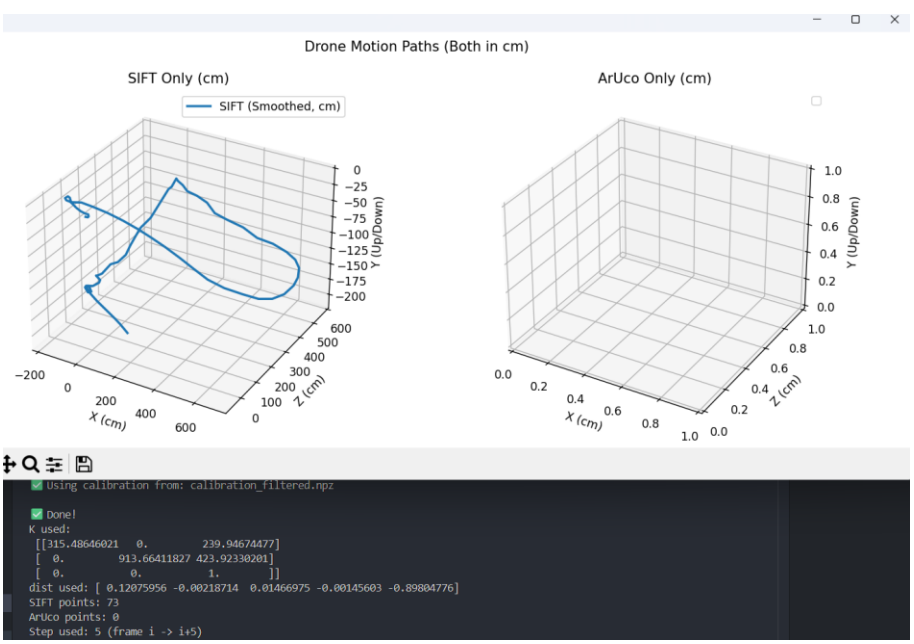

Figure 10.1: SIFT trajectory (scaled) vs ArUco trajectory (ground truth) in centimeters.

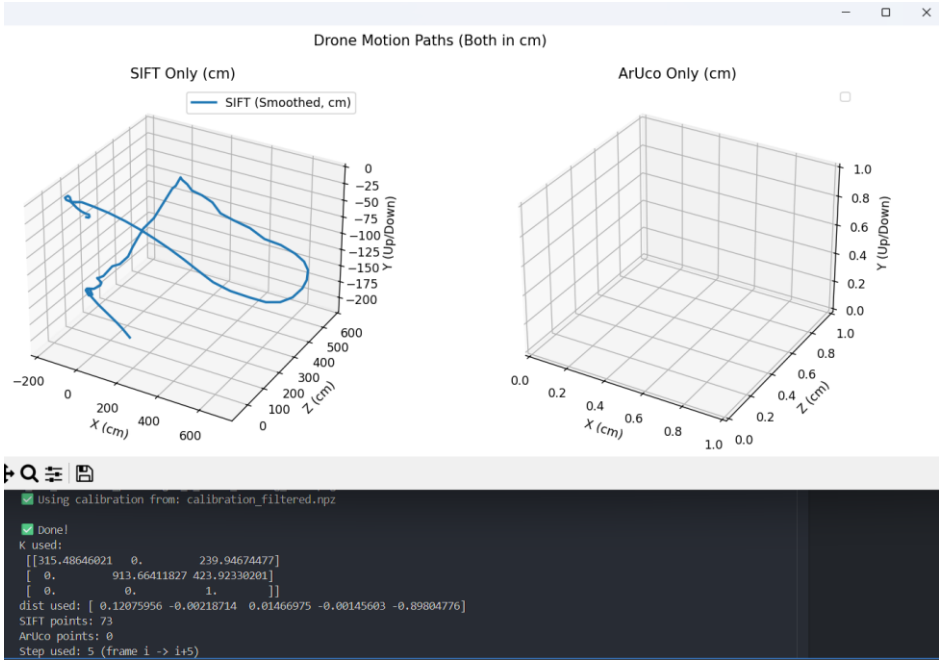

Figure 10.2: Example of missing ArUco detections (marker not detected in some frames).

- We detect the marker and run estimatePoseSingleMarkers.
- We invert marker->camera to get camera position in the marker coordinate system.
- We plot SIFT trajectory (smoothed) together with ArUco trajectory (ground truth).

Sometimes ArUco was empty (no detections). We saw that this happens when the marker is: too small in the image, blurry, overexposed, partially outside the frame, or the dictionary does not match the printed marker.

7) Camera calibration - what we tried and what we used in the end

At first we used a guessed camera matrix K built from the frame size. But we wanted better results, so we tried to calibrate the camera using a ChArUco board video.

- We recorded a calibration video and tried to detect ChArUco corners.
- We repeatedly got: accepted frames = 0 (not enough detected corners).

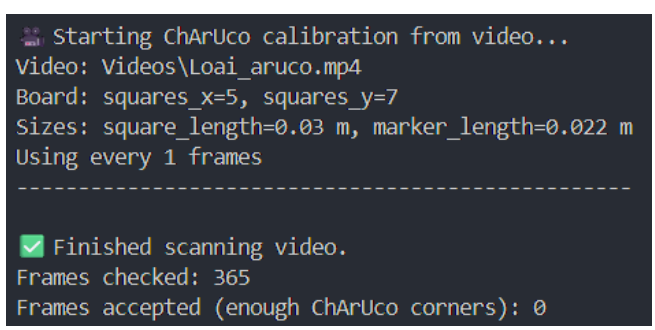

Figure 11.1: First ChArUco calibration attempt failed - no valid frames were accepted.

```
🎥 Starting ChArUco calibration from video...
Video: Videos\Loai_aruco.mp4
Board: squares_x=5, squares_y=7
Sizes: square_length=0.03 m, marker_length=0.022 m
Using every 1 frames
------------------------------------------------

✅ Finished scanning video.
Frames checked: 365
Frames accepted (enough ChArUco corners): 0
```

Figure 11.2: Second ChArUco calibration attempt - still 0 frames accepted.

- In this second attempt we tried several practical tweaks to “force” the detector to accept frames: we scanned every frame (--every_n_frames 1), reduced the required ChArUco corners (min_charuco_corners from 20 down to 6), and used --show to visually debug the detection. Even with these relaxed settings, OpenCV still did not return enough ChArUco corners, so the accepted frame count stayed at 0. This usually means the ChArUco board was not clearly visible in the video (too small, blurred, overexposed, partially out of frame, or wrong board/dictionary).
- Since we could not get a usable calibration from our attempts, we decided to use an existing calibration file (calibration_filtered.npz) and continue the project with it.

Finally, we used an existing file calibration_filtered.npz that already contains:

```
K=\n [[1.32241408e+03 0.00000000e+00 1.00577677e+03]
 [0.00000000e+00 1.32847625e+03 6.16388480e+02]
 [0.00000000e+00 0.00000000e+00 1.00000000e+00]]
\ndist=\n [[ 0.12075956 -0.00218714  0.01466975 -0.00145603 -0.89804776]]
```

Figure 12.1: Contents of calibration_filtered.npz (K and distortion coefficients).

```
Rotation Matrix R:
[[ 0.99575638  0.0665479  -0.06356572]
 [-0.06842378  0.99726888 -0.02780228]
 [ 0.06154193  0.03203371  0.99759031]]

Translation Vector t:
[[-0.6917293 ]
 [-0.18303341]
 [ 0.69857666]]
```

Figure 12.2: Two-frame pose estimation result when using the calibration file (R and t).

```
Rotation Matrix R:
[[ 0.99550429 -0.01477444 -0.09355703]
 [ 0.00761155  0.99704354 -0.07646068]
 [ 0.0944101   0.07540482  0.99267358]]

Translation Vector t:
[[-0.97762741]
 [ 0.20982655]
 [-0.01474683]]
```

Figure 12.3: Two-frame pose estimation result without calibration (shows that the output changes).

- K (3x3) - camera intrinsic matrix
- dist (1x5) - distortion coefficients

Example load (used in our scripts):

```
data = np.load('calibration_filtered.npz')
K = data['K']
dist = data['dist']
```

Using the real K and dist made the pose estimation more consistent, because the geometry model matches the real camera.

8) Back from video to 2 frames (final submission script)

After we tried the video pipeline and used it to improve our matching and stability, we came back to the original requirement: estimating the relative pose between two frames only. So we built a clean final script called pose_estimator_2frames.py that takes 2 images, uses SIFT + Essential Matrix (RANSAC), and outputs the Rotation matrix R and Translation vector t.

```
Rotation Matrix R:
[[ 0.99575638  0.0665479  -0.06356572]
 [-0.06842378  0.99726888 -0.02780228]
 [ 0.06154193  0.03203371  0.99759031]]

Translation Vector t:
[[-0.6917293 ]
 [-0.18303341]
 [ 0.69857666]]
```

Figure 13: Final submission script output (prints only Rotation matrix R and translation vector t).

- Input: --img1 <frame1.jpg> --img2 <frame2.jpg>
- Optional: --calib calibration_filtered.npz (loads K + dist for better accuracy)
- Output: Rotation matrix R (3×3) + Translation vector t (3×1, direction / unscaled)

This is the final function we submit. It uses the same logic we tested in the video version (feature matching → inliers → essential matrix → recoverPose), but now focused on the exact 2-frame task.

9) How to run pose_estimator_2frames.py (command line)

• Example (**with calibration**):

```
python pose_estimator_2frames.py --img1 "img1.jpg" --img2 "img2.jpg" --calib “calibration.npz”
```

• Example (**without calibration**, the script estimates K from image size):

```
python pose_estimator_2frames.py --img1 "img1.jpg" --img2 "img2.jpg"
```

The script prints R and t directly to the terminal.

10) Summary (what we learned)

We started with a simple two-frame pose estimator, and the first results were often unstable because wrong matches and weak geometry created noisy motion. After that, we moved to a video pipeline so we could see the behavior over many frame pairs and improve it step by step. First, we improved matching (KNN + Lowe ratio test) so we keep fewer but more reliable correspondences.

Then we switched from ORB to SIFT, which gave more stable features in our scenes. We also tightened outlier rejection using RANSAC and an inlier threshold, and we skipped updates when the inlier count was too low (instead of adding “garbage” motion).

Finally, we applied smoothing to reduce small remaining noise in the plotted trajectory.
To deal with the scale problem, we added ArUco as a real-world reference whenever detections were available, and we tested the effect of real camera calibration (K + distortion) to make the geometry more consistent.

In the end, we took what worked best from the video experiments and extracted it back into the final two-frame script (the submission script) that outputs only **R** and **t**.